

Aula 16 — Autenticação JWT no React

Módulo: ReactJS + .NET

Carga horária: 4 horas

Projetos base:

- **API:** Aula11.Api (com JWT já implementado)
- **Front:** aula13-react (React + Vite)

Objetivo da aula:

Implementar autenticação JWT completa no React, entendendo armazenamento seguro do token, rotas privadas (guards), refresh token (conceito aplicado) e logout seguro, utilizando Axios Interceptors.

PARTE 1 — TEORIA (≈ 2 horas)

1 Token Storage (Armazenamento do Token)

◆ Onde armazenar o JWT?

Opção	Uso	Observação
localStorage	SPA simples	Vulnerável a XSS
sessionStorage	Sessão curta	Menos persistente
Memory (Context)	Profissional	Requer refresh token
Cookie HttpOnly	Ideal	Backend controla

👉 Nesta aula (didática):

- Access Token em **memory (Context)**

- Refresh Token (conceitual)

📌 Regra de mercado:

Tokens **não devem** ficar expostos permanentemente no browser.

2 Private Routes (Guards de Rota)

◆ Problema

Evitar acesso direto a rotas protegidas:

```
/admin /produtos
```

◆ Solução

Criar um **componente de rota protegida** que:

- Verifica autenticação
 - Redireciona para `/login` se não autenticado
-

3 Refresh Token (Conceito)

◆ Por que usar?

- Access Token expira rápido
- Refresh Token gera novo Access Token sem novo login

Fluxo:

```
Request → 401 → /refresh → novo accessToken → retry request
```

📌 Implementação completa envolve:

- Tabela de refresh tokens
 - Revogação
 - Device binding
(ideal para aula avançada / pós).
-

🍼 PARTE 2 — PRÁTICA (≈ 2 horas)

🎯 Objetivo Prático

Implementar no React:

- Login JWT
 - Contexto de autenticação
 - Guards de rota
 - Interceptor Axios
 - Logout seguro
-

1 Criando o Contexto de Autenticação

📁 src/context/AuthContext.jsx

```
import { createContext, useState, useEffect } from "react"; import api from "../services/api"; export const AuthContext =
createContext(); export function AuthProvider({ children }) { const [token, setToken] = useState(null); const [usuario,
setUsuario] = useState(null); async function login(email, password) { const response = await api.post("/auth/login", { email,
password }); const accessToken = response.data.accessToken; setToken(accessToken); setUsuario({ email }); } function logout() {
setToken(null); setUsuario(null); } return ( <AuthContext.Provider value={{ token, usuario, login, logout, isAuthenticated: !!token }} >
{children} </AuthContext.Provider> ); }
```

2 Envolvendo a Aplicação

 main.jsx

```
import React from "react"; import ReactDOM from "react-dom/client"; import App from "./App"; import { AuthProvider } from
"./context/AuthContext"; ReactDOM.createRoot(document.getElementById("root")).render( <AuthProvider> <App /> </AuthProvider> );
```

3 Axios Interceptor (JWT automático)

 src/services/api.js

```
import axios from "axios"; import { getAuthToken } from "./tokenStore"; const api = axios.create({ baseURL:
"https://localhost:5001/api/v1" }); api.interceptors.request.use(config => { const token = getAuthToken(); if (token) {
config.headers.Authorization = `Bearer ${token}`; } return config; }); export default api;
```

 src/services/tokenStore.js

```
let token = null; export function setAuthToken(t) { token = t; } export function getAuthToken() { return token; } export function
clearAuthToken() { token = null; }
```

 Integrar no AuthContext :

```
setAuthToken(accessToken);
```

4 Tela de Login

 components/Login.jsx

```
import { useContext, useState } from "react"; import { AuthContext } from "../context/AuthContext"; import { useNavigate } from
"react-router-dom"; function Login() { const { login } = useContext(AuthContext); const navigate = useNavigate(); const [email,
```

```
setEmail] = useState(""); const [password, setPassword] = useState(""); async function submit(e) { e.preventDefault(); await login(email, password); navigate("/produtos"); } return ( <form onSubmit={submit}> <input placeholder="Email" value={email} onChange={e => setEmail(e.target.value)} /> <input type="password" placeholder="Senha" value={password} onChange={e => setPassword(e.target.value)} /> <button type="submit">Entrar</button> </form> ); } export default Login;
```

5 Private Route (Guard)

 components/PrivateRoute.jsx

```
import { useContext } from "react"; import { Navigate } from "react-router-dom"; import { AuthContext } from "../context/AuthContext"; function PrivateRoute({ children }) { const { isAuthenticated } = useContext(AuthContext); return isAuthenticated ? children : <Navigate to="/login" />; } export default PrivateRoute;
```

6 Configurando as Rotas

 App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom"; import Login from "../components/Login"; import Produtos from "../components/Produtos"; import PrivateRoute from "../components/PrivateRoute"; function App() { return ( <BrowserRouter> <Routes> <Route path="/login" element={<Login />} /> <Route path="/produtos" element={<PrivateRoute> <Produtos /> </PrivateRoute> } /> </Routes> </BrowserRouter> ); } export default App;
```

7 Logout Seguro

 components/Header.jsx

```
import { useContext } from "react"; import { AuthContext } from "../context/AuthContext"; function Header() { const { logout } = useContext(AuthContext); return ( <button onClick={logout}> Sair </button> ); } export default Header;
```

- ✓ Limpa token da memória
 - ✓ Redireciona usuário
 - ✓ Sessão encerrada corretamente
-

8 Fluxo Final Implementado

Login → JWT recebido → Token armazenado em memória → Axios envia token automaticamente → Rotas protegidas → Logout limpa sessão

✓ Resultado Esperado

- ✓ Login JWT funcional
 - ✓ Token protegido (memory)
 - ✓ Guards de rota
 - ✓ Interceptors Axios
 - ✓ Logout seguro
 - ✓ SPA profissional com segurança
-

📌 Atividade Avaliativa

👉 Desafio prático:

1. Persistir login após refresh (refresh token conceitual)
2. Tratar erro 401 no interceptor
3. Redirecionar automaticamente para /login
4. Criar rota /admin protegida

5. Proteger por role (claim no JWT)